Context Engineering

Context Engineering is the practice of deliberately designing, structuring, and managing what information goes into an agent's context window at each step. So the agent has exactly what it needs to reason and act well, nothing more, or nothing less.

It is the evolution of the "Prompt Engineering" instead of just a good prompt, you are architecting the entire information flow the agent sees.

Baseline, an LLM can only see what is in the context window, so the agent's intelligence is directly limited by what you put in front of it.

- **Bad Context** --> Confused/bad agent --> Wrong tool calls --> Hallucinations
- **Good Context** --> Focused agent, correct decision --> Reliable output.

Limitation

We cannot simply put everything in the context window, but the agent tasks are long, multi-step, and involve lots of information.

Context engineering aim to

1. **What to include:** Not everything is relevant at every step.
   For example, a coding agent does not need the full 10000 line codebase, just the relevant file and function.
2. **What to compress or summarize:** Older conversation turns, past tool results, and completed steps can be summarized instead of kept verbatim.
3. **What to retrieve (RAG)**
   Instead of pre-loading all knowledge, pull in relevant information dynamically based on the current step.
4. **What order to put things in**:
   LLMs pay more attention to content at the beginning and end of context.

StoreBackend

Credit to: Krish Naik

```
# --- StoreBackend: persistent, cross-thread memory ---
# WHY: files/memory live in a LangGraph BaseStore, NOT in one conversation's state.
# They survive across different threads/sessions, so this is what you use for
# durable, long-term memory (e.g. per-user preferences the agent should recall
# next week). Here InMemoryStore is used for the demo; swap in a persistent store
# (e.g. Postgres) for real persistence.
from deepagents import create_deep_agent
from deepagents.backends.store import StoreBackend
```

```
from langgraph.store.memory import InMemoryStore
from langgraph.checkpoint.memory import MemorySaver

store = InMemoryStore()
# Seed durable memory into the store once. Namespace + key identify the file.
store.put(("memories",), "AGENTS.md", create_file_data(agents_md))

store_agent = create_deep_agent(
    model="openai:gpt-5.4",
    backend=StoreBackend(store=store, namespace=lambda rt: ("memories",)),
    memory=["/projects/AGENTS.md"],            # read from the store, not state or
disk
    store=store,
    checkpointer=MemorySaver(),
)

# Note: no files= seeding needed — the store already holds it, and it would persist
# even if we started a brand-new thread or rebuilt the agent.
result = store_agent.invoke(
    {"messages": [{"role": "user", "content": "What's in your memory? Who are
you?"}]},
    config={"configurable": {"thread_id": "store-demo-1"}},
)
print(result["messages"][-1].content)
```

FilesystemBackend

```
# --- FilesystemBackend: read files/memory straight from DISK ---
# WHY: the agent's files and memory map to real files on disk under root_dir.
# Nothing to seed — if AGENTS.md exists on disk, memory just loads it. Best for
# project context (AGENTS.md, docs, code) that already lives in your repo.
from deepagents import create_deep_agent
from deepagents.backends.filesystem import FilesystemBackend
from langgraph.checkpoint.memory import MemorySaver

# root_dir = the projects/ folder, so paths are resolved relative to it.
# AGENTS.md sits directly inside projects/, so the memory path is just "AGENTS.md".
# virtual_mode=True confines the agent to root_dir (blocks '..' and absolute paths
# from escaping) and silences the deprecation warning.
fs_agent = create_deep_agent(
    model="openai:gpt-5.4",
    backend=FilesystemBackend(root_dir="projects", virtual_mode=True),
    memory=["AGENTS.md"],                          # -> projects/AGENTS.md
    checkpointer=MemorySaver(),
)

# Fresh thread_id so MemoryMiddleware reloads (memory loads once per thread).
result = fs_agent.invoke(
    {"messages": [{"role": "user", "content": "What's in your memory? Who are
```

```
you?"}]},
    config={"configurable": {"thread_id": "fs-demo-2"}},
)
print(result["messages"][-1].content)
```